

Python Pandas: The Ultimate Guide (Zero to Hero)

This guide covers the essence of Data Analysis and Manipulation, ranging from basic setup to advanced filtering.

1. Introduction: What is Pandas?

- **Definition:** Pandas is a Python library used for handling and analyzing **Structured Data** (data with rows and columns).
- **Use Case:** It is an essential tool for performing **EDA (Exploratory Data Analysis)** in Machine Learning.
- **Supported Formats:** It allows loading data from various formats like CSV, Excel, JSON, and pickle files.

2. Environment Setup (How to Start Working)

- **VS Code & Jupyter:** Create a new folder and create a `.ipynb` (Jupyter Notebook) file within it.
 - **Kernel:** To run the code, attach the notebook to the correct Python kernel and test if it is working correctly.
-

3. Reading Data (Loading Data)

Pandas converts data into a **DataFrame**, which represents the data in a tabular form (like an Excel sheet).

- **CSV File:** `df = pd.read_csv('file.csv')`.
 - **Excel File:** `df = pd.read_excel('file.xlsx', sheet_name='Sheet1')`.
 - **Online URL:** If the data is online, you can read it directly using the link:
`pd.read_csv('link')`.
-

4. Basic Data Exploration (Understanding the Data)

Once the data is loaded, use these basic commands to understand it:

- **`df.head()` / `df.tail()`:** To view the first few and last few rows of the DataFrame.
- **`df.shape`:** To get the dimensions (total number of rows and columns).

- **df.info()**: Provides a summary including column names, data types, and non-null counts.
 - **df.describe()**: Provides a statistical summary of numerical columns (Mean, Min, Max, Standard Deviation, Count, Percentiles).
+1
 - *Note*: Use the **include** parameter to analyze object-type columns as well.
 - **df.columns**: Extracts the list of all column names.
-

5. Selection & Filtering (Choosing and Sorting Data)

A. Basic Selection

- **Columns**: To select specific columns, use bracket notation: `df[['Col1', 'Col2']]` or dot notation: `df.Col1`.
+1
- **Simple Filter**: Filtering rows based on a condition: `df[df['Score'] > 50]`.
- **Multiple Conditions**: Use logical operators **&** (AND) and **|** (OR) to combine conditions.
+1

B. Advanced Selection: loc vs iloc 🎯

This is the most important topic for precise data selection.

- **loc (Label-based)**: Uses names (Labels).
+1
 - *Syntax*: `df.loc[row_label, column_label]`.
 - *Slicing*: The last element is **included** (e.g., `0:2` gives rows 0, 1, and 2).
 - *Usage*: Best used for filtering (e.g., `df.loc[df['Age'] > 18]`).
- **iloc (Integer-based)**: Uses Position (Index Numbers).
 - *Syntax*: `df.iloc[row_index, col_index]`.
 - *Slicing*: Like standard Python lists, the last element is **excluded** (e.g., `0:2` gives only rows 0 and 1).

C. Special Filters

- **isin()**: Used when you need to filter based on multiple values in a single column.
 - **between()**: Used to filter data falling within a specific range.
-

6. Data Manipulation (Modifying Data)

- **New Column:** Creating a new feature from existing data: `df['Total'] = df['Maths'] + df['Science'] + 1`
 - **apply() Function:** Used to apply custom logic or functions to the DataFrame.
 - **map() Function:** Used to map values in a column to other values based on a dictionary.
 - **replace():** Used for string manipulation to modify column values.
 - **Sorting:** To sort the data: `df.sort_values(by='Date', ascending=False)`.
-

7. Data Cleaning & Aggregation

- **Unique Values:**
 - `unique()`: Finds unique values in a column.
 - `nunique()`: Counts the number of unique values.
 - `value_counts()`: Counts the frequency of each unique value.
 - **Pivot Tables:** Summarizes data based on multiple columns (similar to Excel pivot tables).
 - **Cleaning:**
 - `drop()`: Used to remove rows or columns. The `inplace` parameter makes the change permanent.
 - `astype()`: Used to convert data types (e.g., from String to Float).
-

8. Saving Data (Saving Your Work)

- Save the DataFrame to a file after analysis: `df.to_csv('final.csv', index=False)`.
 - *Note:* The `index=False` argument prevents saving the DataFrame index as an extra column in the CSV file.
-

🌟 Special Notes & Pro-Tips (From Me)

1. **Series vs DataFrame:** When you extract a column using single brackets `[ ]`, it is a **Series** (1D). However, using double brackets `[ [ ] ]` results in a **DataFrame** (2D table). Remember this distinction.
2. **Memory Tip:** Always check `df.info()`. If a column has numbers but the data type is 'Object', use `astype()` to convert it and save memory.

3. **Inplace True/False:** Most Pandas operations return a new copy of the data. If you want the changes to apply to the original DataFrame, you must use `inplace=True` (e.g., in `drop`).
4. **Practice:** The next lecture covers complex operations, so make sure to practice `loc`, `iloc`, and `apply` thoroughly.